

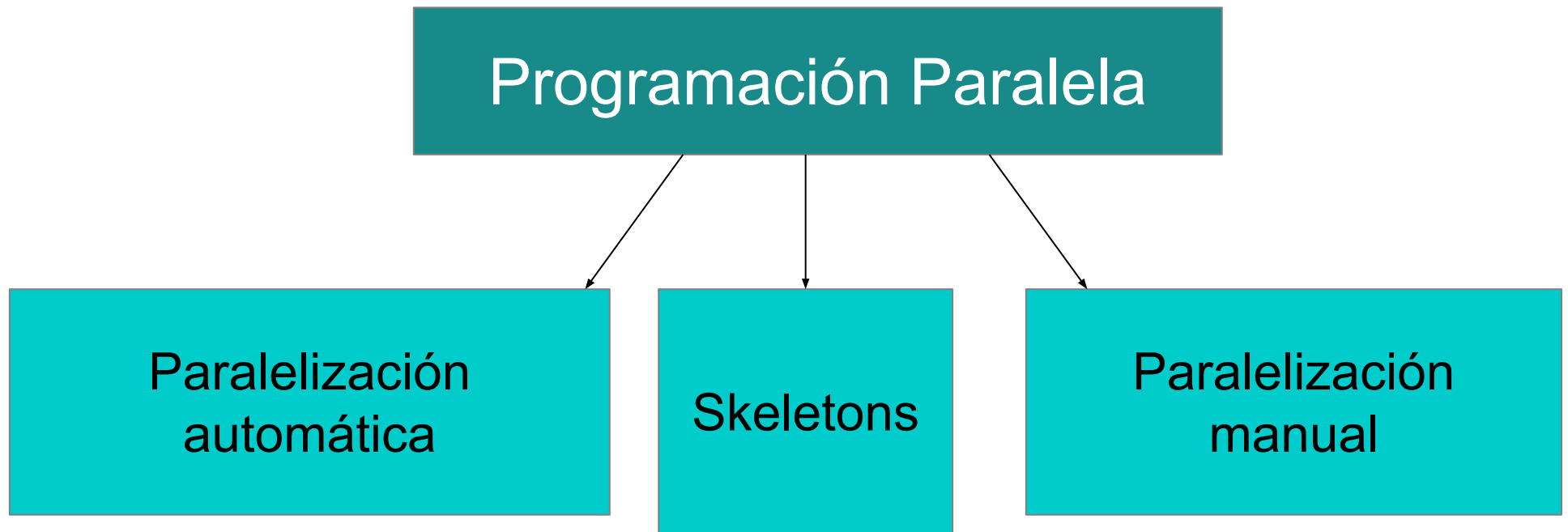
Unidad 5

Diseño de programas paralelos

- Paralelización automática, skeletons y manual.
- Aspectos claves de la programación paralela manual.
 - Análisis del problema y del posible programa serie.
 - Estrategia de descomposición del problema (particionamiento).
 - Distribución del trabajo (scheduling)
 - Mapping
 - Comunicaciones
 - Dependencias de datos entre tareas y sincronización.
- Entrada y salida.
- Paradigmas de programación paralela (*actividad de lectura*).

Paralelización automática,
skeletons y manual

Paralelización automática vs. manual



Paralelización automática

- Diseñar y desarrollar programas paralelos en forma **manual** lleva mucho tiempo, es complejo, propenso a errores, y es un proceso iterativo.
- La **paralelización automática** intenta transformar un programa serie en uno paralelo.
- El usuario podría concentrarse en **qué** calcular y no en **cómo** hacer el cálculo.
- Para ser aceptable, la paralelización **automática** debería generar código con **rendimiento competitivo** con el código manual escrito por un experto.
 - La paralelización automática puede funcionar muy mal para ciertas aplicaciones serie.
- Puede involucrar: Compilador y Sistema de soporte en tiempo de ejecución.
 - Los compiladores normalmente aplican paralelización automática mediante el uso de instrucciones vectoriales cuando sea posible (en casos triviales).

Paralelización automática

- Desde sus inicios hasta la actualidad:
 - Se inicia en los 70' con la **vectorización** automática basada en análisis de dependencias.
 - Luego el foco pasó a máquinas MIMD de **memoria compartida**, con cierto éxito paralelizando **bucles** (loops).
 - Cuando aparecen las máquinas de memoria distribuida e híbridas, la complejidad aumentó considerablemente y la paralelización automática tuvo poco éxito, sobre todo para desarrollar soluciones **escalables**.
- Un ejemplo de paralelización automática para memoria compartida:

Decoupled Software Pipelining (DSWP)

<http://liberty.princeton.edu/Projects/AutoPar/>

Paralelización manual

- Actualmente, la paralelización manual está dominando.
- Se programa para una arquitectura de sistema específica.
- Tendencia: los investigadores se han orientado preferentemente a desarrollar estrategias, basadas en lenguajes, que intentan **reducir la carga al programador**.
 - Hacer los lenguajes más simples y utilizables para distintas arquitecturas.
 - Proveer al programador de construcciones paralelas típicas.
- El presente curso se enfoca principalmente en la paralelización manual.

Skeletons

- Los skeletons son un tipo de programación de alto nivel para la computación paralela y distribuida, con inicios en 1989.
- Los patrones comunes de cómputo y comunicaciones en paralelo se pueden empaquetar en construcciones del tipo “framework/template”, y generalmente son presentados sin referencia al paralelismo explícito. Tales construcciones son “esqueletos”, que tienen una estructura definida y a la cual se debe agregar el detalle del cómputo específico a la aplicación.
- Podrían construirse patrones complejos mediante la combinación de los básicos.
- Tiene implementaciones eficientes para ciertas arquitecturas.
- Ocultan complejidad, son fáciles de utilizar.
- Pueden no adaptarse a todas las aplicaciones.
- Pueden lograr menor rendimiento que una paralelización manual.

Análisis del problema
y del posible programa serie

Análisis del problema

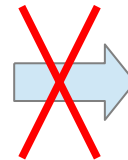
- ¿El problema es lo suficientemente grande para que justifique su programación en paralelo?
- El proceso de diseño comienza en el **dominio del problema** con los elementos de diseño relevantes al problema que debe ser resuelto (átomos, flujos de fluidos, etc.). Así, el primer paso es entender el dominio del problema



- Un programa paralelo solo será exitoso si el problema contiene concurrencia explotable (múltiples tareas que pueden ejecutarse al mismo tiempo)
- El diseño comienza buscando concurrencia de elementos del problema

Análisis del problema: el programa serie

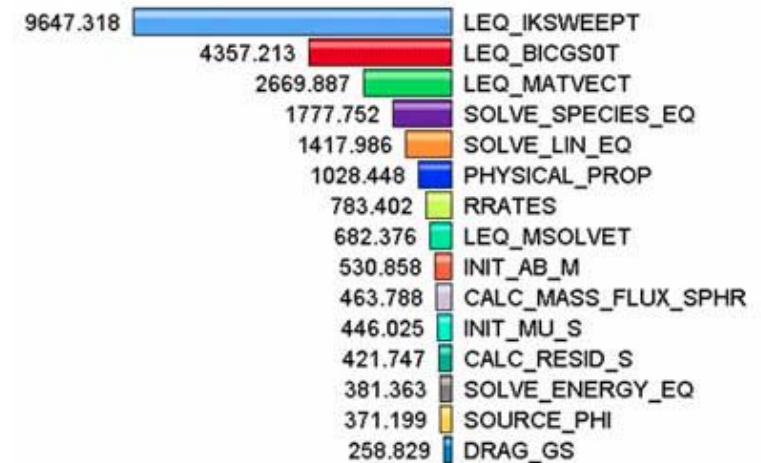
- Si se cuenta con un programa serie, es útil entender el código existente.
- Sin embargo, no se obtiene una aplicación paralela de alto rendimiento a partir del código serie sino planteando una nueva solución al problema.



Análisis del problema:

Análisis del posible programa serie

- Enfocarse en paralelizar las partes de cómputo más significativas. ¿Se logrará el objetivo de velocidad paralelizando esas partes? (Ley Amdahl, Gustafson).
- Identificar inhibidores de paralelismo: dependencias de datos.
- Identificar cuellos de botella:
 - ¿Hay secciones desproporcionadamente lentas que pueden evitar que la ejecución paralela avance? Ej. la E/S.
 - ¿Puede reestructurarse el programa o utilizarse otros algoritmos que reduzcan los cuellos de botella?
- Buscar algoritmos alternativos
 - Hay disponibles librerías con funciones matemáticas y físicas optimizadas: IBM-Parallel ESSL, Intel-MKL, AMD-AMCL, BLAST, etc.



Estrategia de descomposición del
problema o Particionamiento

Estrategias de descomposición del problema

- El proceso de **descomposición** (o particionamiento) de un problema consiste en dividir un problema en partes discretas de **trabajo** que pueden ser distribuidas a múltiples tareas.
 - Las tareas son unidades de cómputo definidas por el programador.
 - La reducción de tiempo se logra por la ejecución simultánea de tareas.

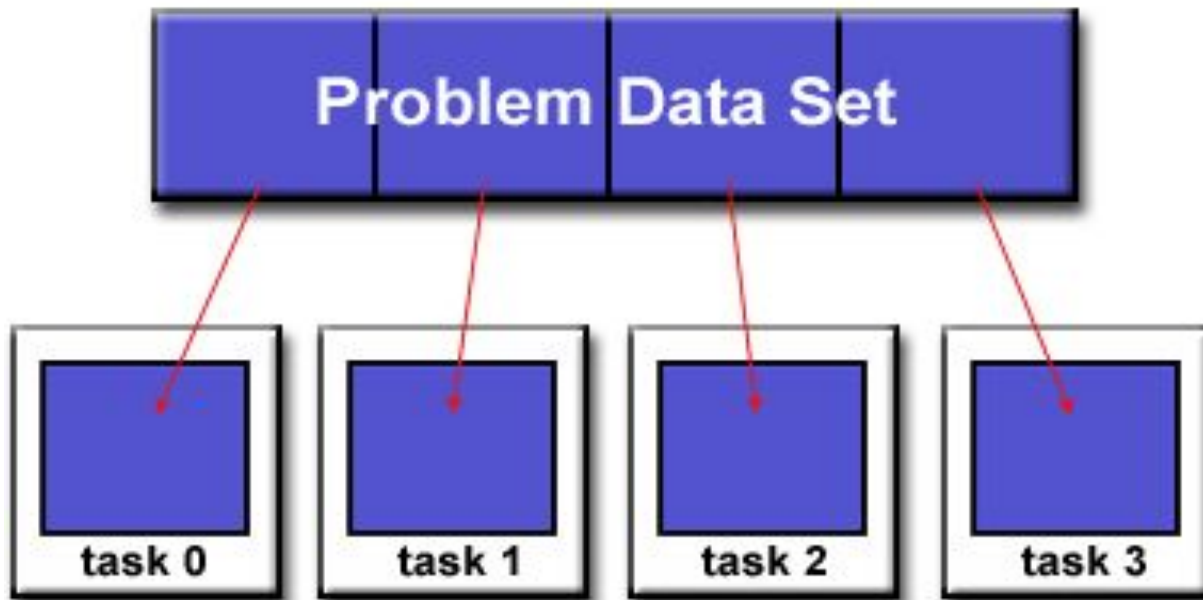


- Hay dos estrategias de descomposición básicas para dividir el trabajo en tareas paralelas:
 - descomposición por **dominio (data parallelism)**
 - descomposición **funcional (task parallelism)**

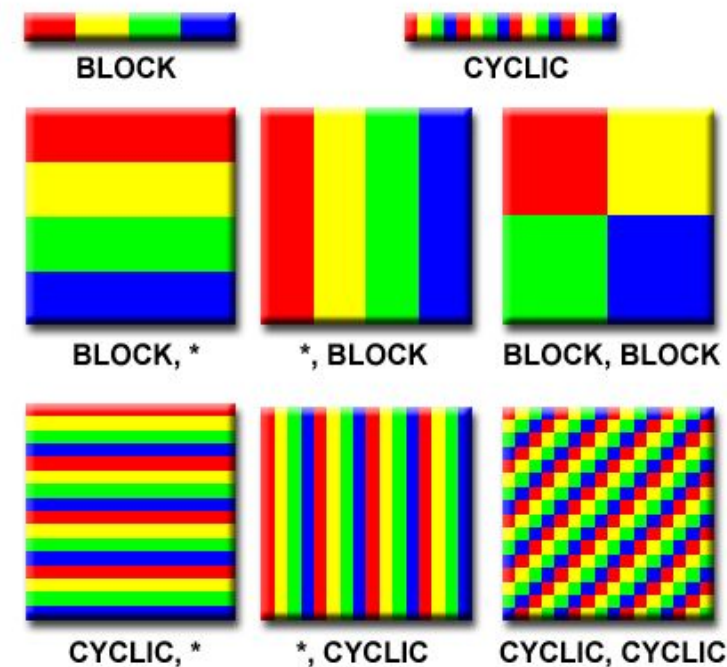
Estrategias de descomposición del problema

Descomposición por dominio

Se descomponen los datos asociados al problema en múltiples regiones, y cada tarea paralela trabajará sobre una región.



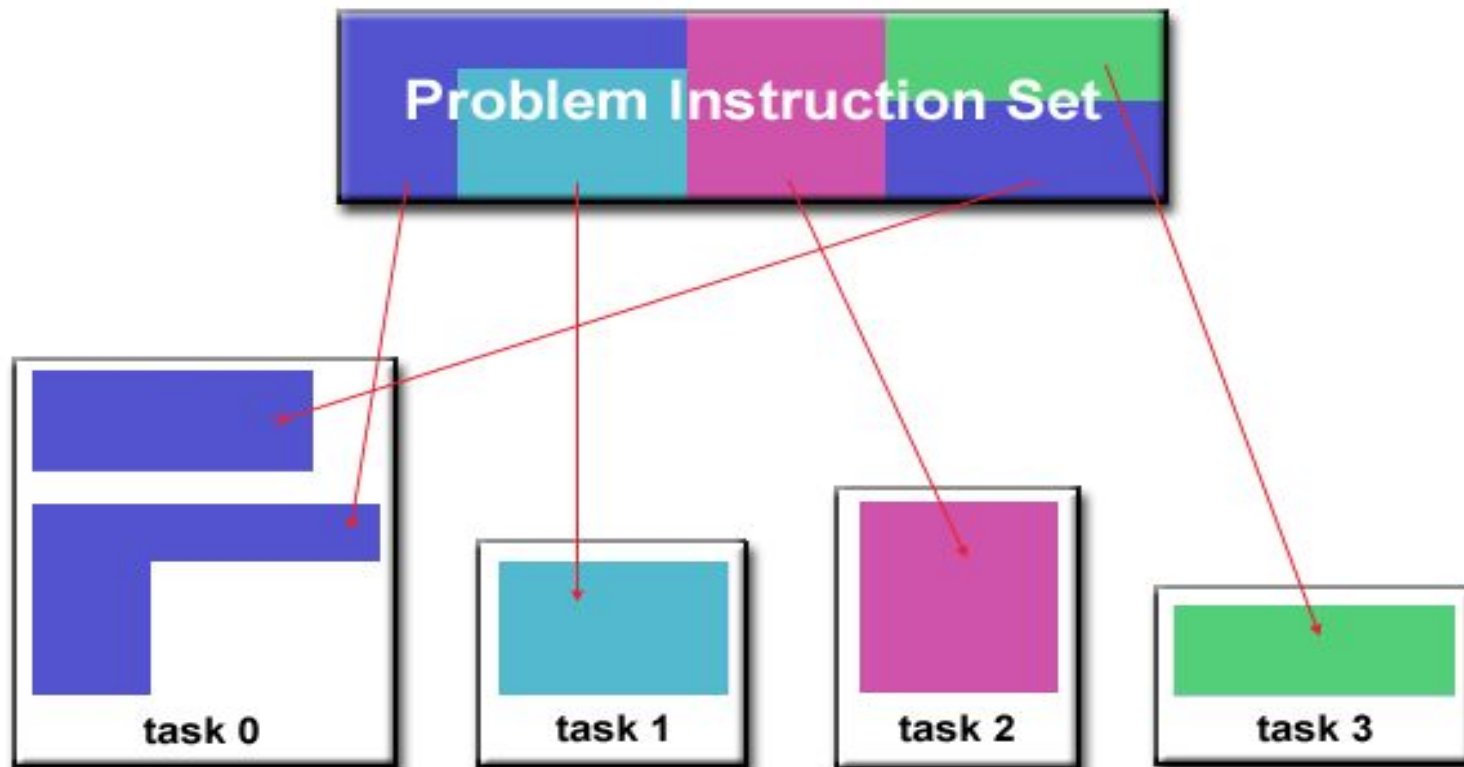
Diferentes descomposiciones de dominio:



Estrategias de descomposición del problema

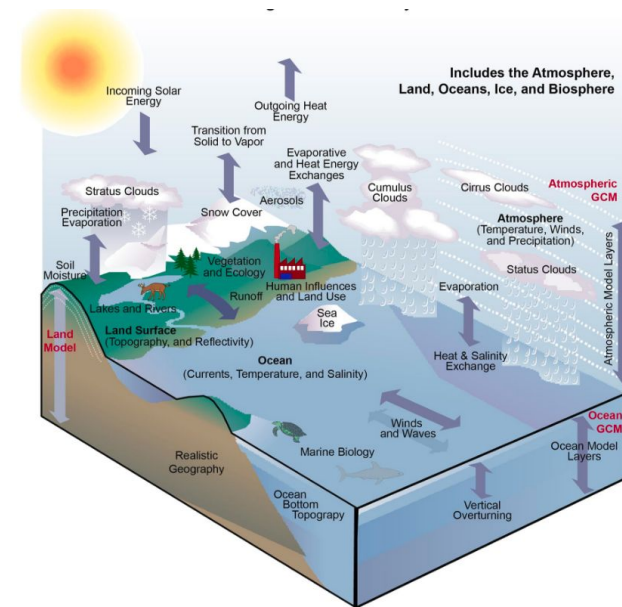
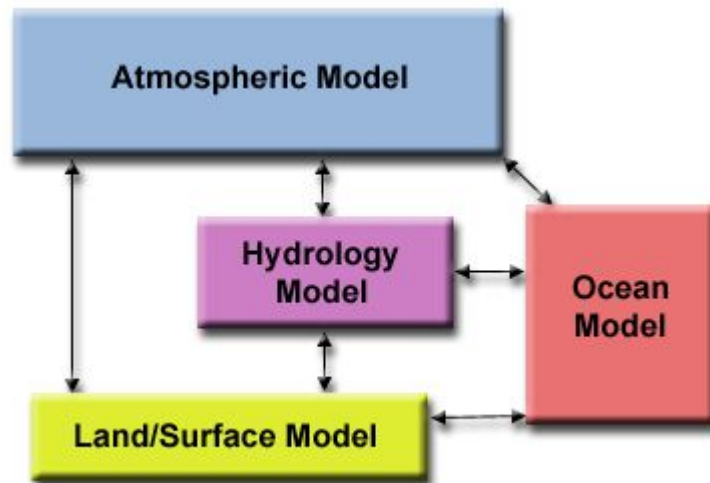
Descomposición funcional

- Se enfoca en el cómputo en vez de los datos manipulados por el cómputo.
- El problema se descompone de acuerdo al trabajo que debe realizarse.
- Cada tarea realiza una parte del trabajo completo.



Estrategias de descomposición del problema

- La descomposición funcional se utiliza normalmente en problemas que presentan una división funcional natural.
 - Ejemplo: modelado del clima



- La descomposición por dominio es más usada porque escala naturalmente.
- La descomposición funcional presenta normalmente un grado de paralelismo pequeño.
- Es común y natural combinar ambas estrategias.

Estrategias de descomposición del problema

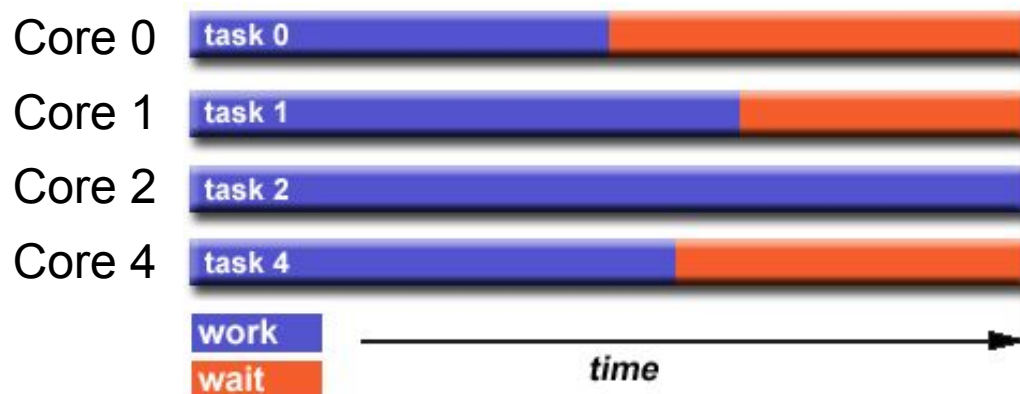
Algunas preguntas que podríamos hacernos

- **¿Las tareas son de tamaño similar?**
 - Si no, puede ser difícil de asignar a cada procesador igual cantidad de trabajo.
- **¿El número de tareas aumenta al aumentar la cantidad de procesadores?**
 - De no lograrse, la aplicación no podrá adaptarse al uso de más procesadores, ya sea para tratar con un tamaño de problema mayor o para resolver más rápido un problema.

Distribución del trabajo

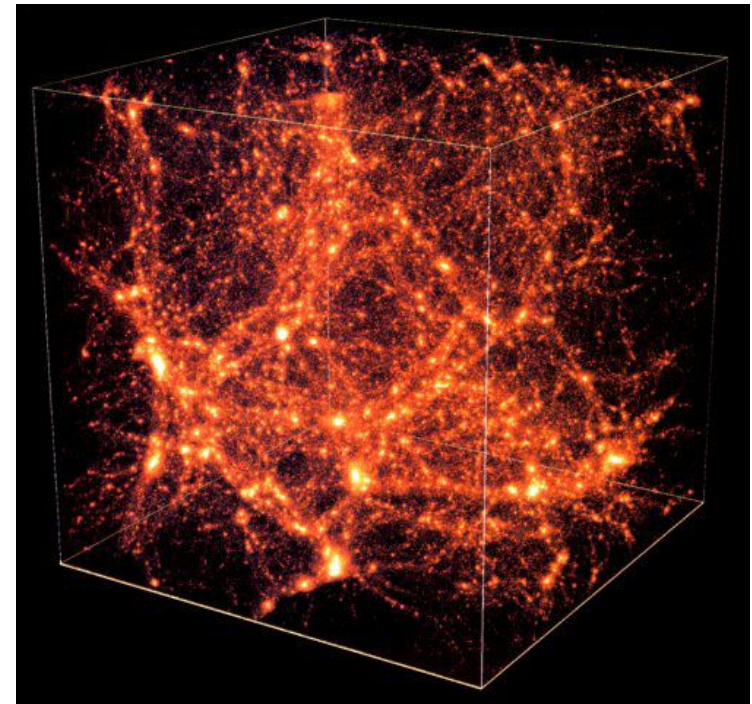
Distribución del trabajo

- La **distribución del trabajo** o simplemente **planificación** (*scheduling*) es la actividad de **asignar tareas a procesos o threads**.
- El **balance de carga** se refiere a la práctica de distribuir aproximadamente igual cantidad de trabajo a los procesadores para evitar que ciertos procesadores queden ociosos esperando a que otros terminen su trabajo.



Distribución del trabajo: estática vs. dinámica

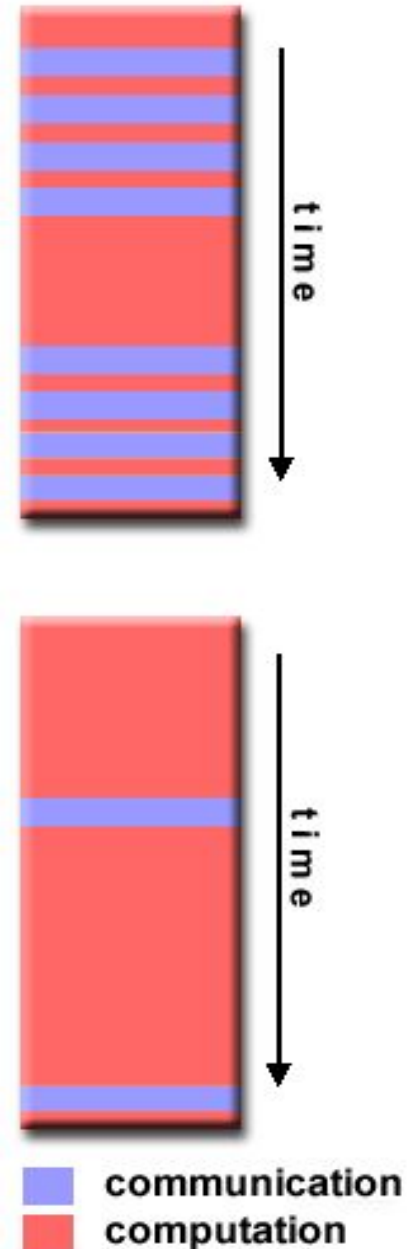
- La distribución del trabajo puede ser:
 - **estática**: la actividad de planificación se realiza antes de comenzar a procesar cualquier parte del trabajo.
 - **dinámica**: la actividad de planificación continúa aún cuando parte del trabajo ya comenzó a procesarse.
- El **balance de carga** que podría lograr una planificación dinámica es superior al de una planificación estática.
- La **sobrecarga** de la gestión del paralelismo requerida para una planificación estática es inferior a la requerida por una dinámica.



Distribución del trabajo dinámica: grano fino vs. grano grueso

El trabajo puede ser distribuido dinámicamente en porciones pequeñas o grandes:

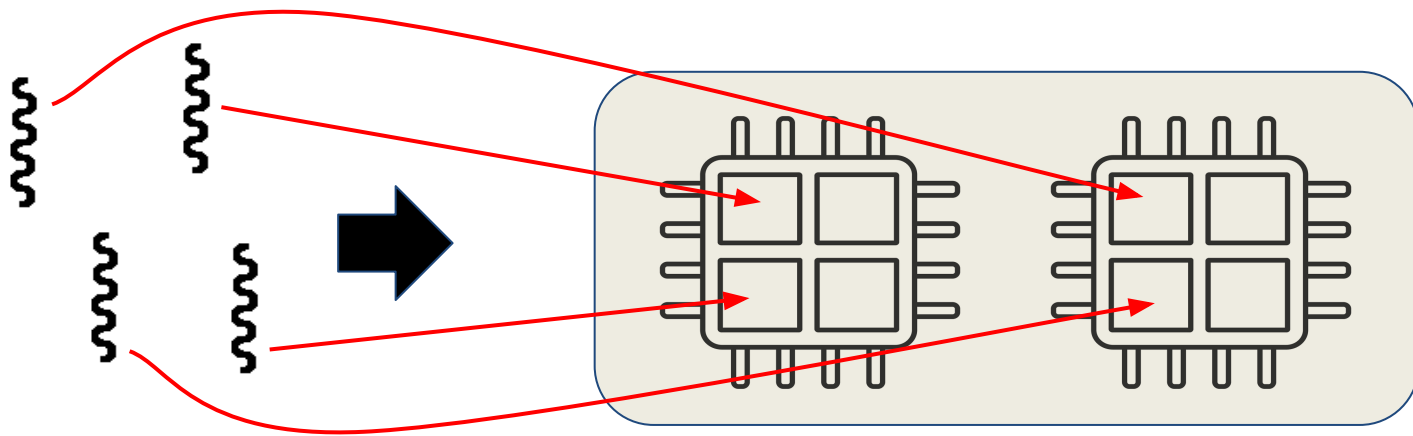
- Si se distribuyen *porciones pequeñas*, el desbalance de carga se reduce pero la sobrecarga por comunicaciones aumenta.
- Si se distribuyen *porciones grandes*, la sobrecarga por comunicaciones se reduce pero el desbalance de carga aumenta.



Mapping

Mapping

- El mapping es la estrategia de asignación de procesos/threads a procesadores.



- Se deben contemplar los requerimientos de comunicaciones entre procesos/threads y la capacidad del sistema de comunicaciones.
- El mapping es normalmente estático pero podría ser dinámico, es decir, cambiar en tiempo de ejecución.
- El mapping idealmente debería estar controlado por la aplicación y no librado a las decisiones del sistema operativo.

Comunicaciones

Definición y necesidad de comunicaciones

- Las tareas paralelas podrían intercambiar datos. El intercambio de datos se denomina “comunicaciones” sin importar el método de intercambio empleado:
 - comunicación explícita por mensajes.
 - comunicación implícita por acceso a variables compartidas.
- La necesidad de comunicaciones depende del problema.
 - Un problema que no requiere comunicaciones se denomina “***embarazosamente paralelo***”.

Costo de las comunicaciones

Las comunicaciones entre tareas siempre implican una sobrecarga:

- empaquetado y transmisión de datos por una red, y posiblemente alguna sincronización entre las tareas involucradas en la comunicación.
- sincronización para accesos a variables compartidas.

Costo de las transmisiones: latencia y ancho de banda

- Latencia es el tiempo requerido para transmitir un mensaje de tamaño mínimo entre dos puntos de una comunicación.
- Ancho de banda es la cantidad de datos que pueden ser comunicados por unidad de tiempo.

latencia 
ancho de banda 

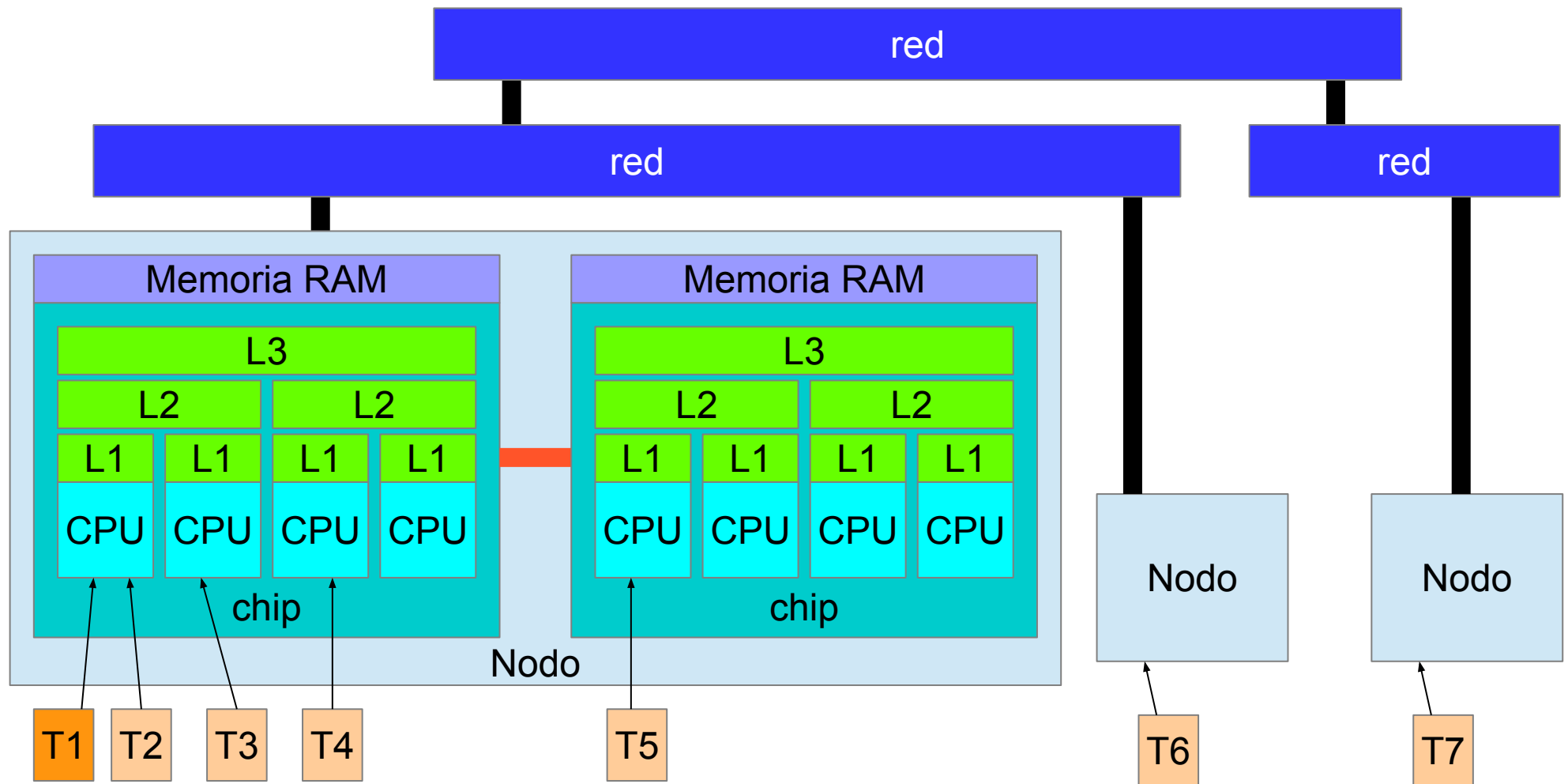


 latencia
 ancho de banda

$$\text{Tiempo para transferir un mensaje de } N \text{ bytes} = \text{latencia} + \frac{N}{\text{ancho de banda}}$$

Enviar muchos mensajes pequeños hace que la latencia domine el retardo de las transferencias. Empaquetar pequeños mensajes en uno más grande aprovecha el ancho de banda y reduce el impacto de la latencia.

Jerarquía de comunicaciones

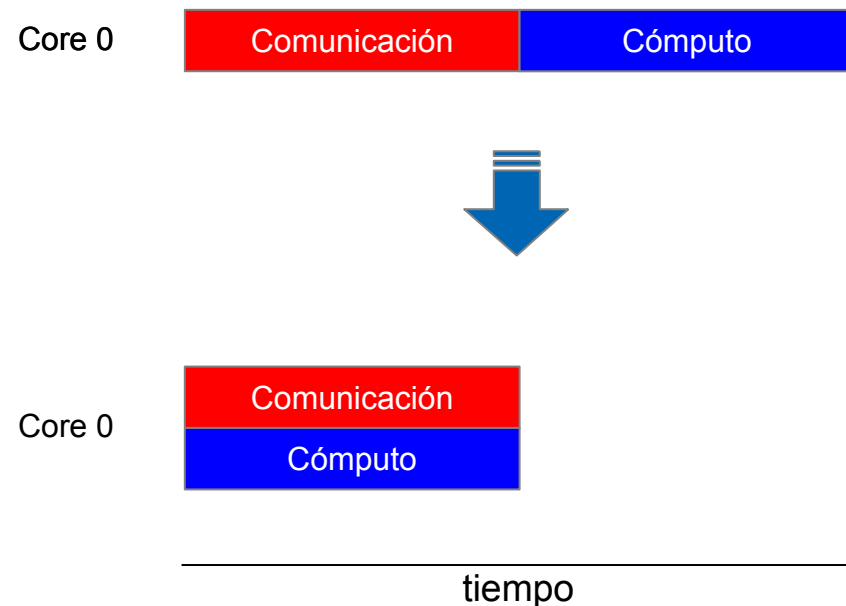


T1 tiene diferentes latencias y anchos de banda con cada uno de los otros threads.

Solapamiento de comunicaciones y cómputo

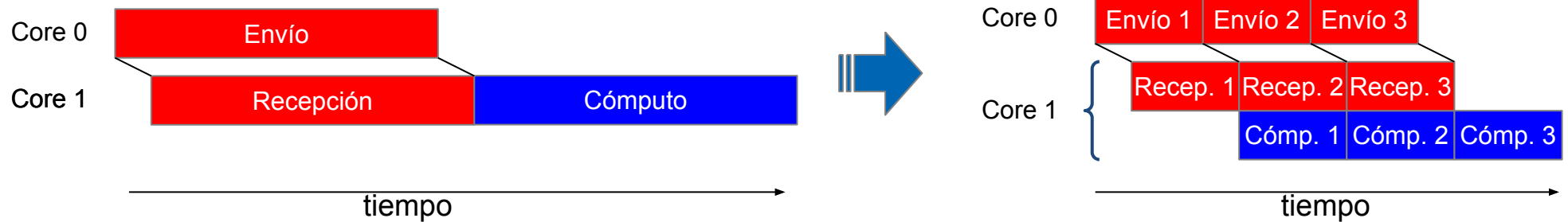
- El objetivo es mantener a las CPUs con trabajo.
- Siempre que sea posible, se computará mientras se llevan a cabo comunicaciones.

Cómputo independiente de la comunicación

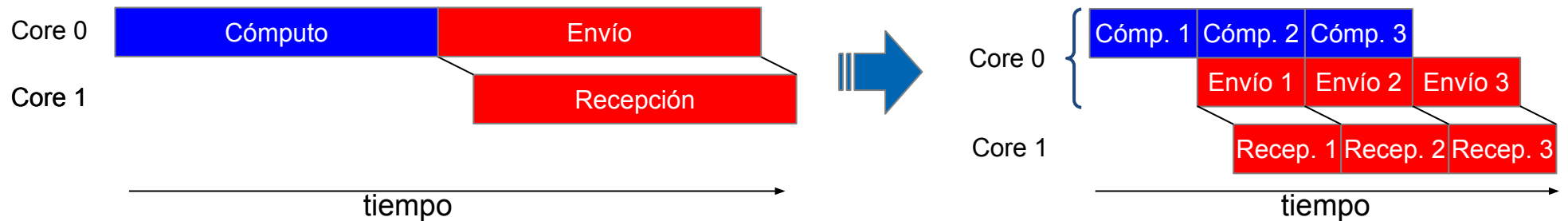


Solapamiento de comunicaciones y cómputo

Cómputo dependiente de la comunicación



Comunicación dependiente del cómputo



Dependencias de datos entre tareas y Sincronización

Las tareas podrían ser independientes y ejecutadas en paralelo sin importar la secuencia temporal. Sin embargo, en muchos programas, las tareas usan datos producidos por otras tareas.

Dependencias de datos entre tareas

- Existe una dependencia de datos entre tareas cuando el orden de ejecución de las tareas afecta al resultado del programa.
- La dependencia de datos se produce por el acceso de **diferentes tareas** a un mismo dato, y cuando al menos uno de esos accesos es de escritura.
- La dependencia de datos es importante para la programación paralela ya que es el principal inhibidor de paralelismo.
- Las “condiciones de Bernstein” (1966) deben cumplirse para que dos tareas se ejecuten en paralelo:

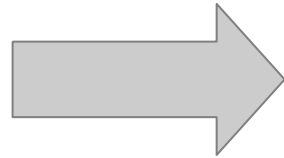
$$\begin{aligned} I_1 \cap O_2 &= \emptyset \\ I_2 \cap O_1 &= \emptyset \\ O_1 \cap O_2 &= \emptyset \end{aligned}$$

I_i es el conjunto de las variables de entrada para una tarea T_i
 O_i es el conjunto de las variables de salida para una tarea T_i

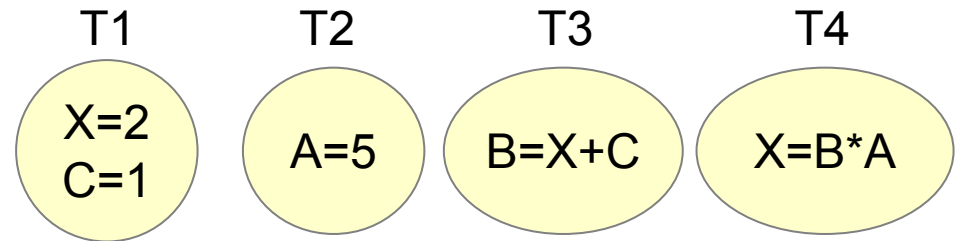
- Es decir, las *dependencia de datos* entre instrucciones (RAW, WAR y WAW), son buscadas entre sentencias de distintas tareas.

Dependencias de datos entre tareas

X=2
C=1
A=5
B=X+C
X=B*A



descomposición
en tareas



código serie

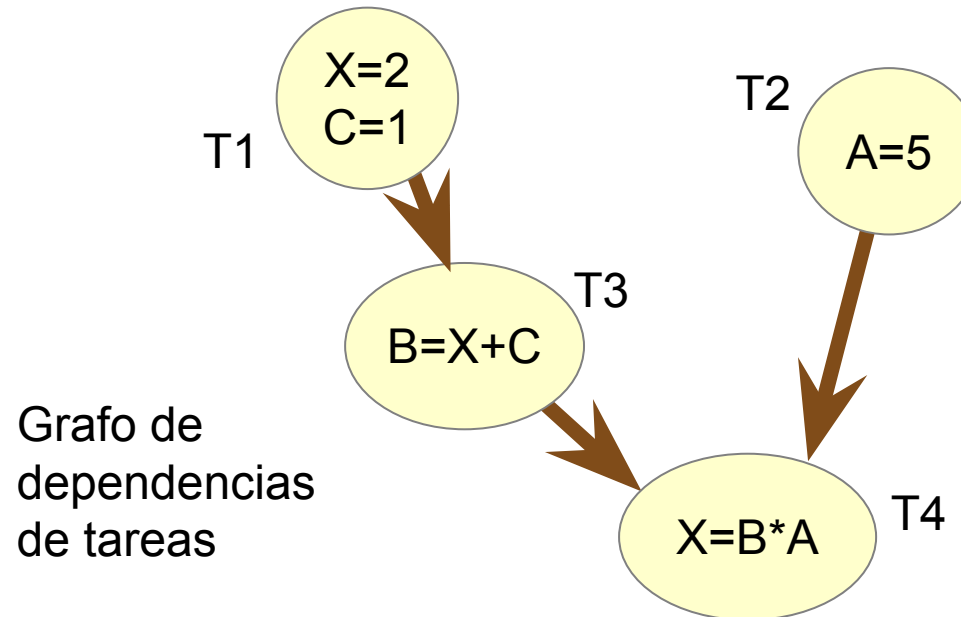
T1 y T2 pueden ejecutarse en paralelo

T1 y T3 no pueden ejecutarse en paralelo ($I_3 \cap O_1 \neq \emptyset$)

...

- $I_1 \cap O_2 = \emptyset$
- $I_2 \cap O_1 = \emptyset$
- $O_1 \cap O_2 = \emptyset$

Condiciones
de Bernstein

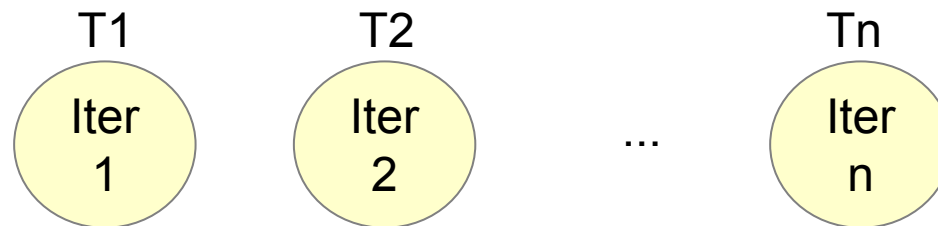


Grafo de
dependencias
de tareas

Dependencias de datos entre tareas

Análisis de dependencias de datos entre iteraciones de bucles

- Los bucles representan una de las fuentes más importantes de paralelismo
- La forma típica de paralelizar bucles es asignar diferentes iteraciones (o bloques de ellas) a diferentes tareas, sin embargo, las iteraciones deben ser independientes



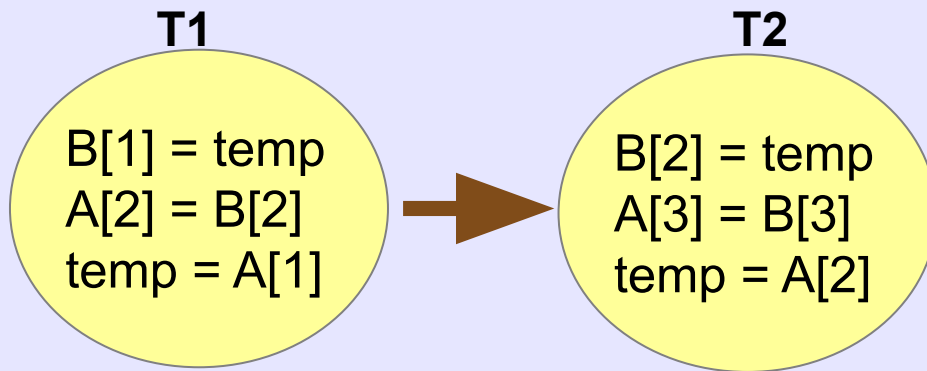
Dependencias de datos entre tareas

Análisis de dependencias de datos entre iteraciones de bucles

Podría desenrollarse una parte del bucle para simplificar el análisis:

```
for (i=1; i<=10; i++) {  
S1:   B[i] = temp;  
S2:   A[i+1] = B[i+1];  
S3:   temp = A[i];  
}
```

```
S1: B[1] = temp  
S2: A[2] = B[2]  
S3: temp = A[1]  
S1: B[2] = temp  
S2: A[3] = B[3]  
S3: temp = A[2]
```



$I_1 \cap O_2 \neq \emptyset$
 $I_2 \cap O_1 \neq \emptyset$
 $O_1 \cap O_2 \neq \emptyset$

No cumple las condiciones de Bernstein

Hay dependencia, por lo tanto una ejecución paralela daría resultados erróneos

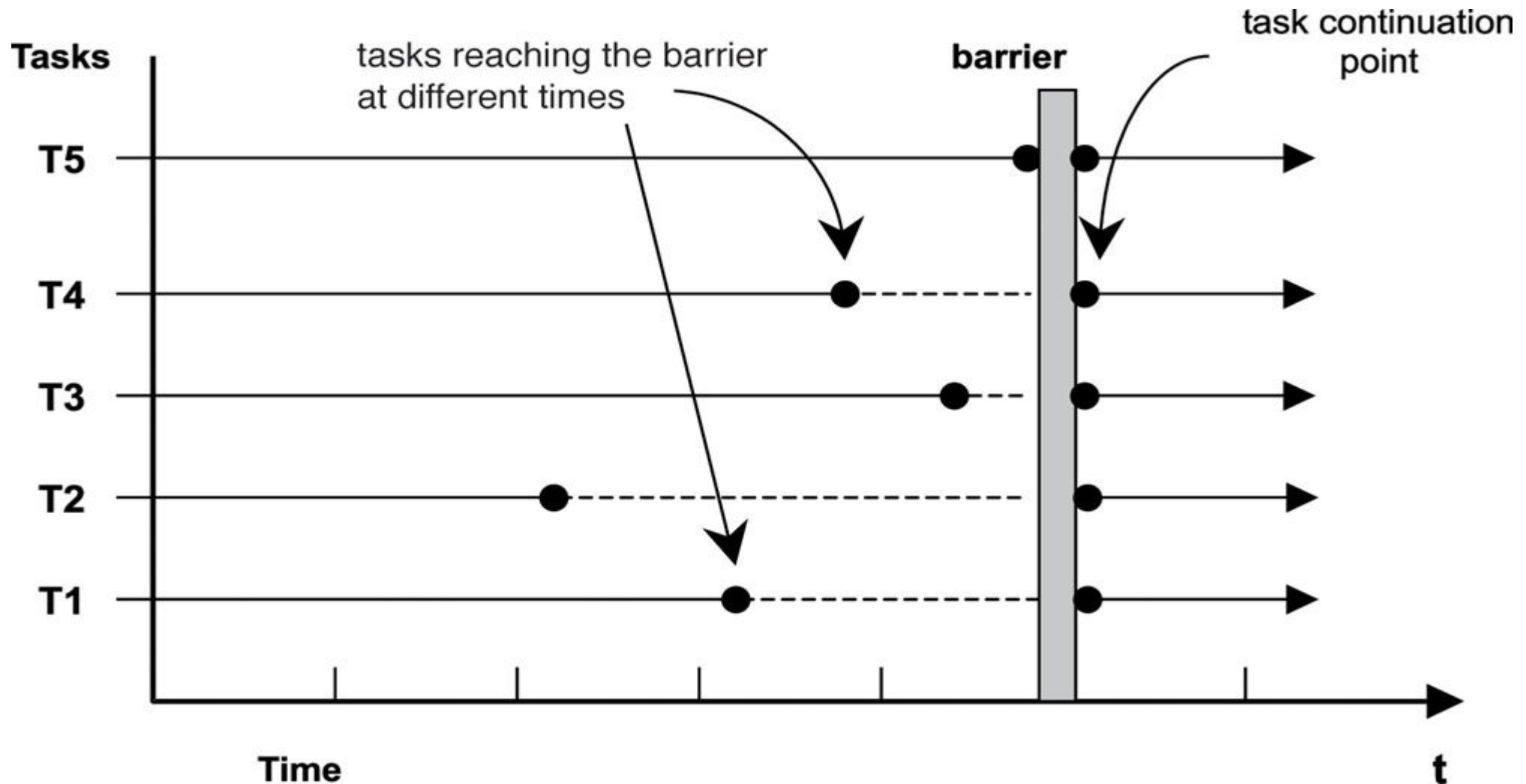
- La dependencia en una misma iteración, S1 – S3 WAR (temp), preocupa al nivel de paralelismo de nivel de instrucciones pero no para paralelizar el bucle.

Sincronización

- Condición de carrera (*race condition*): Múltiples tareas están en condición de carrera si el resultado depende del orden en que se ejecuten las tareas.
- El programa paralelo debe **sincronizarse** para evitar condiciones de carrera.
- Si un programa da resultados diferentes en diferentes ejecuciones, puede deberse a otro factor y no a una condición de carrera. Por ejemplo:
 - Es un programa no determinístico.
 - Es un programa afectado por la no asociatividad de las operaciones de números de coma flotante. Por ejemplo, el resultado de $(A + B) + C$ podría ser (levemente) diferente a $A + (B + C)$.
- Tipos de sincronización:
 - Explícita
 - Barriers (barreras)
 - Locks (bloqueos)
 - Implícita: uso de operaciones de comunicación

Sincronización explícita: Barriers

Ningún proceso puede pasar una barrera hasta que todos la hayan alcanzado



Sincronización explícita: Locks

- Los **locks** se utilizan para acceder **secuencialmente** a una sección crítica (ej. acceso a variables compartidas). Solo una tarea puede acceder por vez. Pueden ser:

- bloqueantes*

```
lock(mutex)
accede al recurso
unlock(mutex)
```

- no bloqueantes*

```
if trylock(mutex)
    accede al recurso
unlock(mutex)
```

- Las **variables de condición** extienden la funcionalidad de los locks permitiendo sincronización basada en valores de datos. Se utilizan en conjunto con los locks.

```
lock(mutex)
cond_signal (condition)
```



```
lock(mutex)
while (alguna condición)
    cond_wait (condition,mutex)
accede al recurso
unlock(mutex)
```

Sincronización implícita

La sincronización implícita puede ocurrir por el **uso de operaciones de comunicación**. En toda comunicación siempre ocurre algún tipo de sincronización. Algunos ejemplos:

- Comunicaciones síncronas

Las tareas emisoras y receptoras se bloquean hasta que finalice la comunicación. Así, ambas tareas están sincronizadas durante la comunicación.

- Comunicaciones asíncronas

Cuando una tarea receptora recibe un mensaje es porque la tarea emisora ya lo transmitió. Entonces, aunque la tarea emisora desconoce el avance de la tarea receptora, la tarea receptora tiene conocimiento respecto al avance de la tarea emisora.

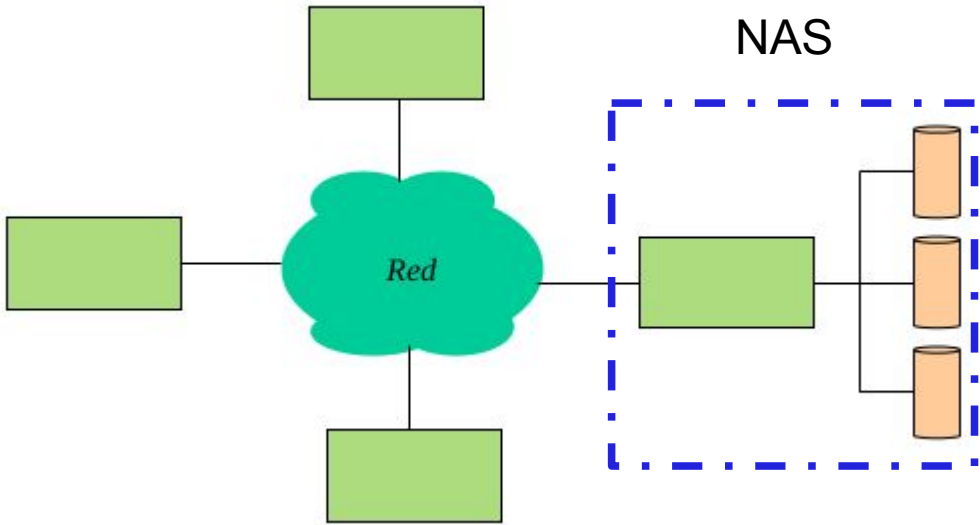
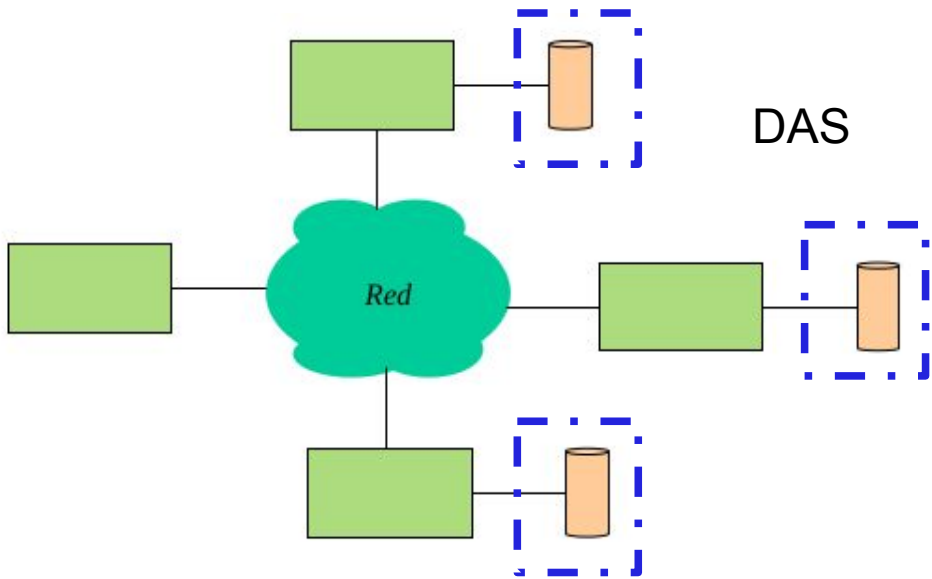
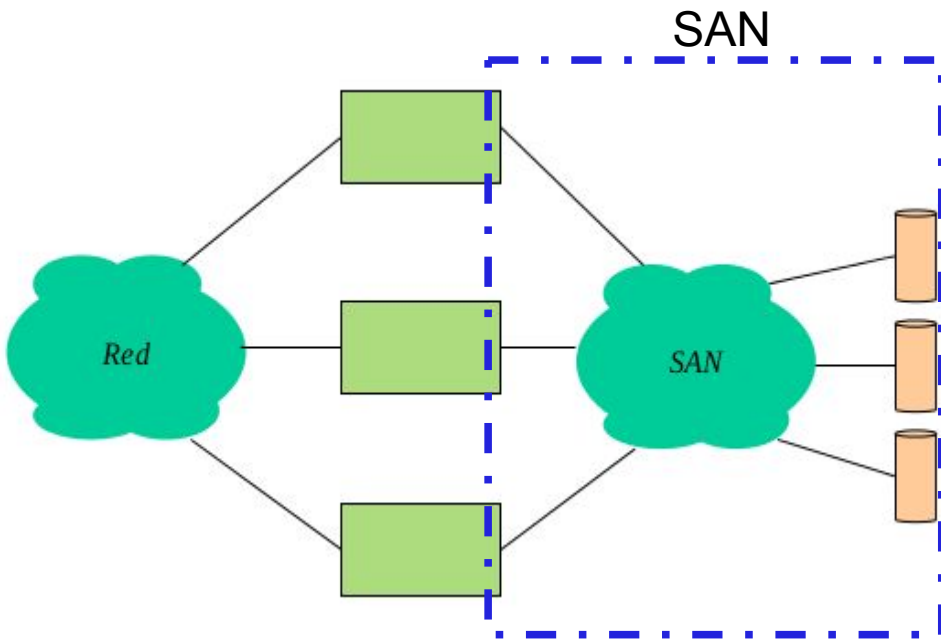
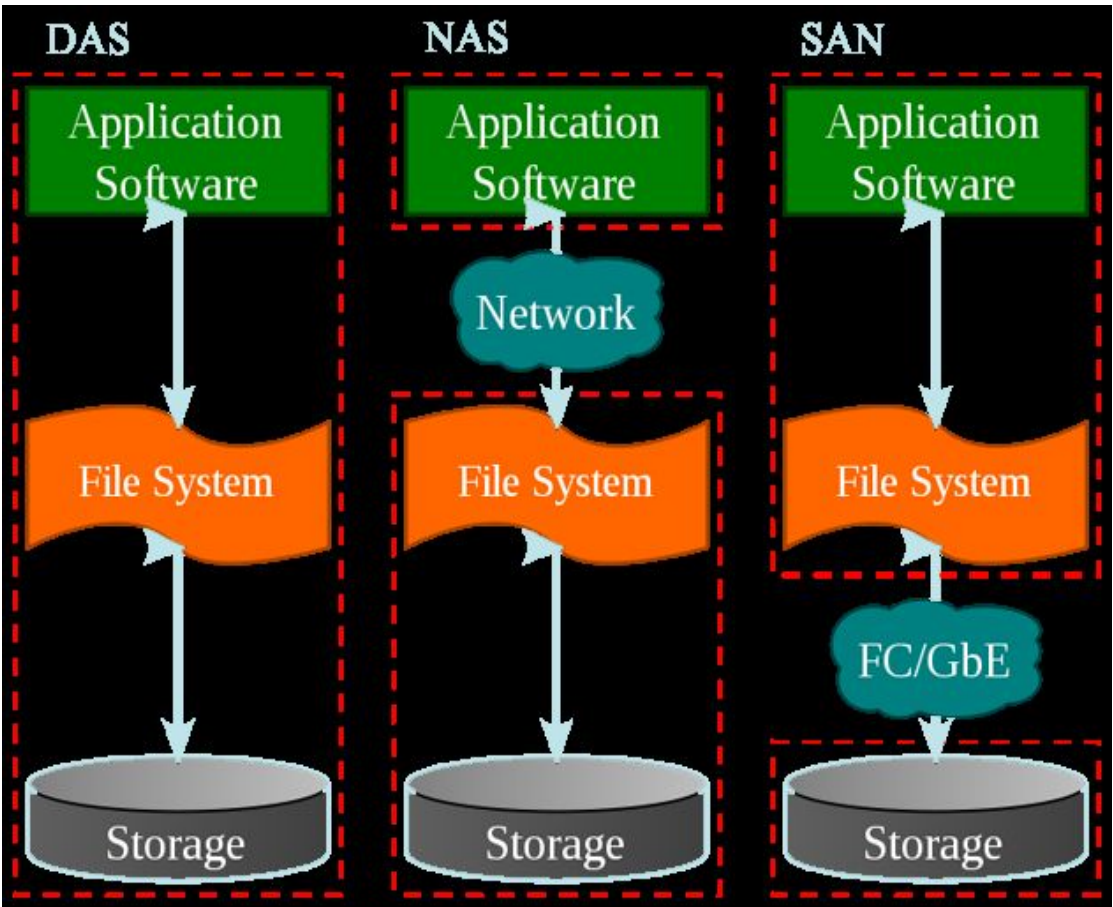
Entrada/Salida

Crisis de Entrada/Salida

- Ciertas aplicaciones utilizan datos de forma masiva
- Crecimiento muy significativo de la capacidad de los discos, pero no de sus prestaciones: ancho de banda y latencia
- Crisis de E/S: desequilibrio entre capacidad de procesamiento y E/S
- Solución: sistemas de archivos distribuidos paralelos

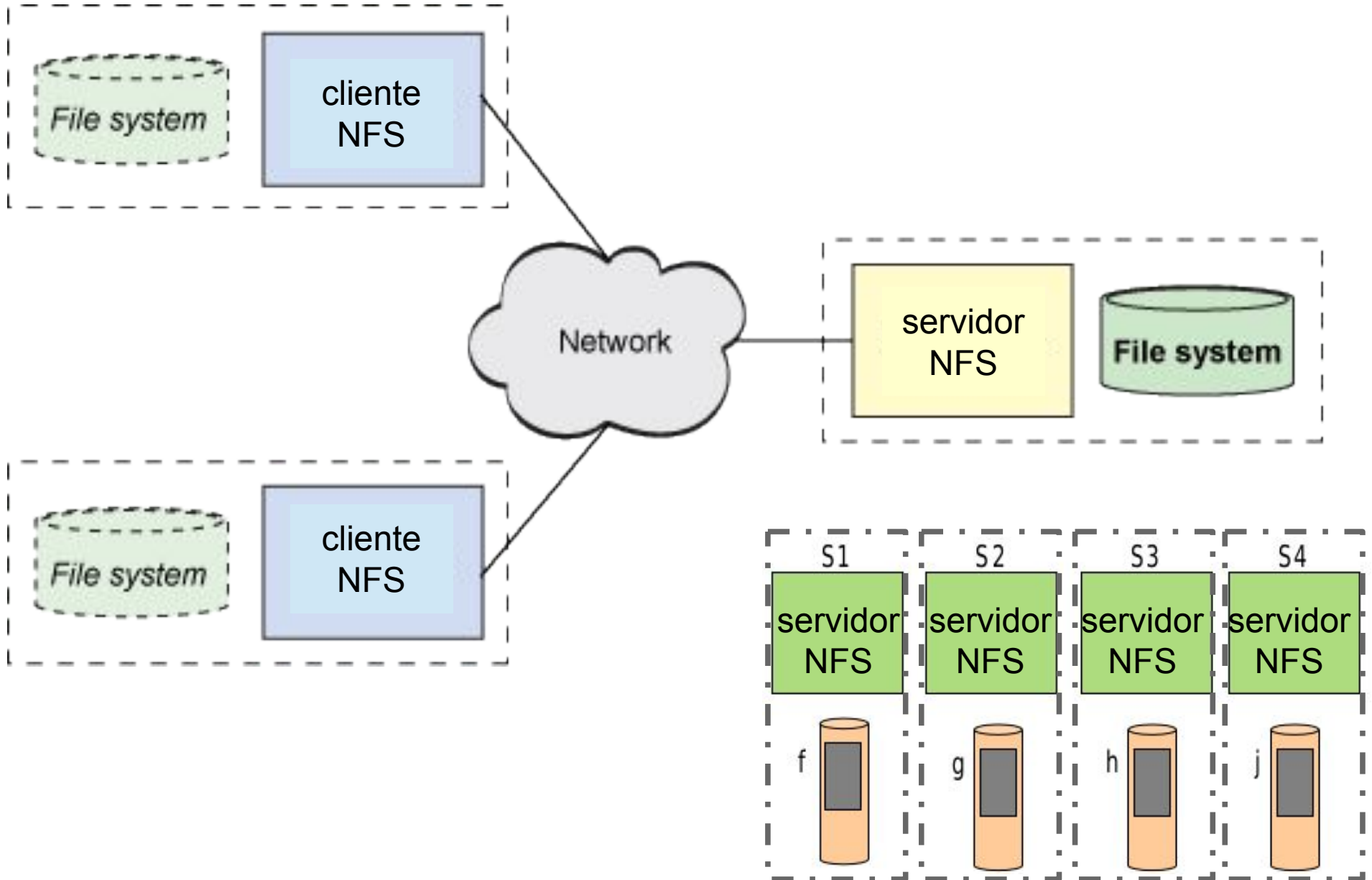
Arquitecturas de almacenamiento

- DAS: Direct-attached storage
 - Solución clásica: dispositivo de almacenamiento conectado a un nodo
- NAS: Network-attached storage
 - Nodo que gestiona uno o más dispositivos
- SAN: Storage Area Networks
 - Red dedicada al almacenamiento
 - Conectividad directa entre nodos y dispositivos

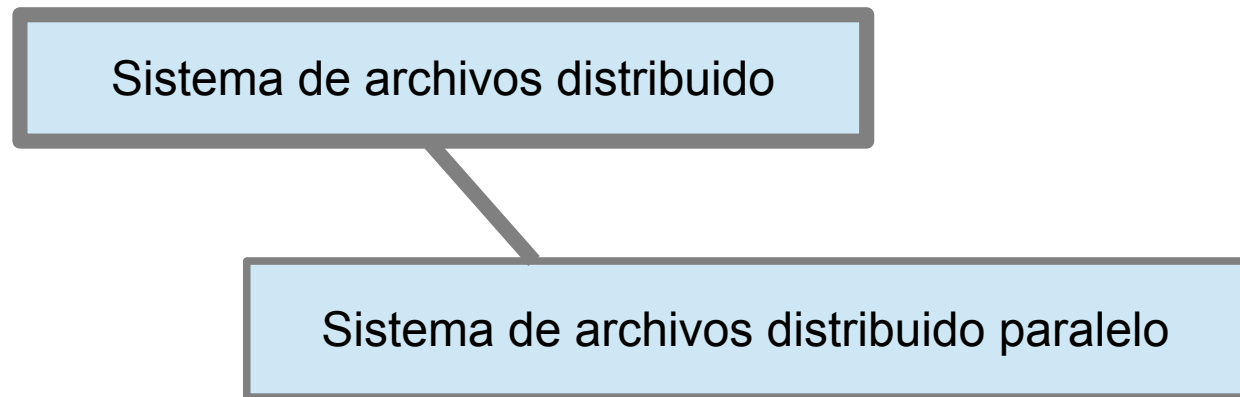


Sistema de archivos distribuido típico

Ejemplo: NFS



Sistemas de archivos distribuidos vs. paralelos

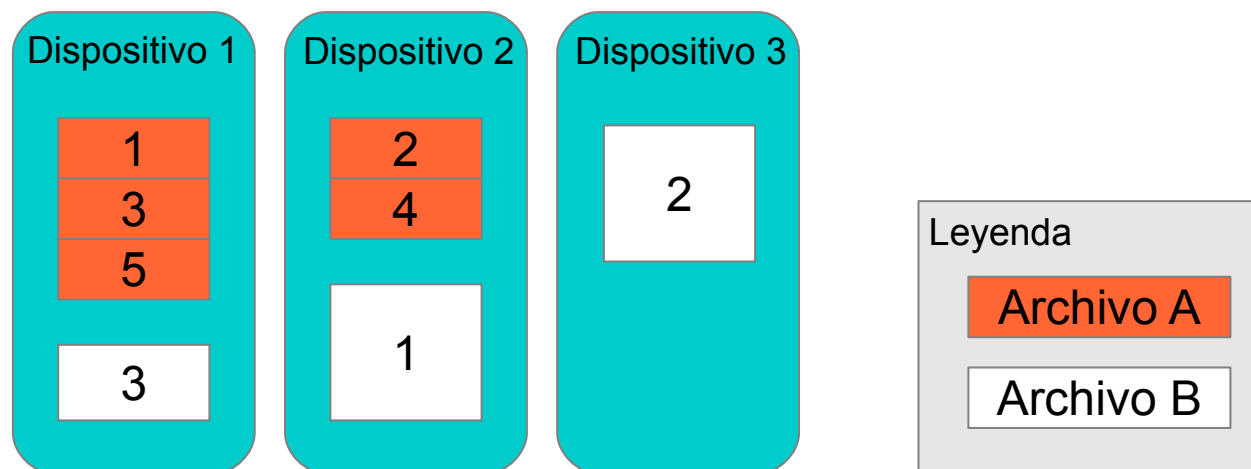


¿Por qué no son adecuados los sistemas de archivos distribuidos típicos para un sistema de cómputo paralelo?

- Un archivo se almacena en un único servidor
 - Puede causar fuertes cuellos de botella.
 - Como no hay paralelismo en el acceso a los archivos, el acceso es lento.

Sistemas de archivos paralelos

Basados en **striping**: Los datos de un mismo archivo están distribuidos entre múltiples discos. El acceso en paralelo permite aprovechar la agregación del ancho de banda de los discos.



Debido a la alta latencia de acceso a discos, es mejor que los datos accedidos sean grandes para aprovechar el ancho de banda del acceso paralelo.

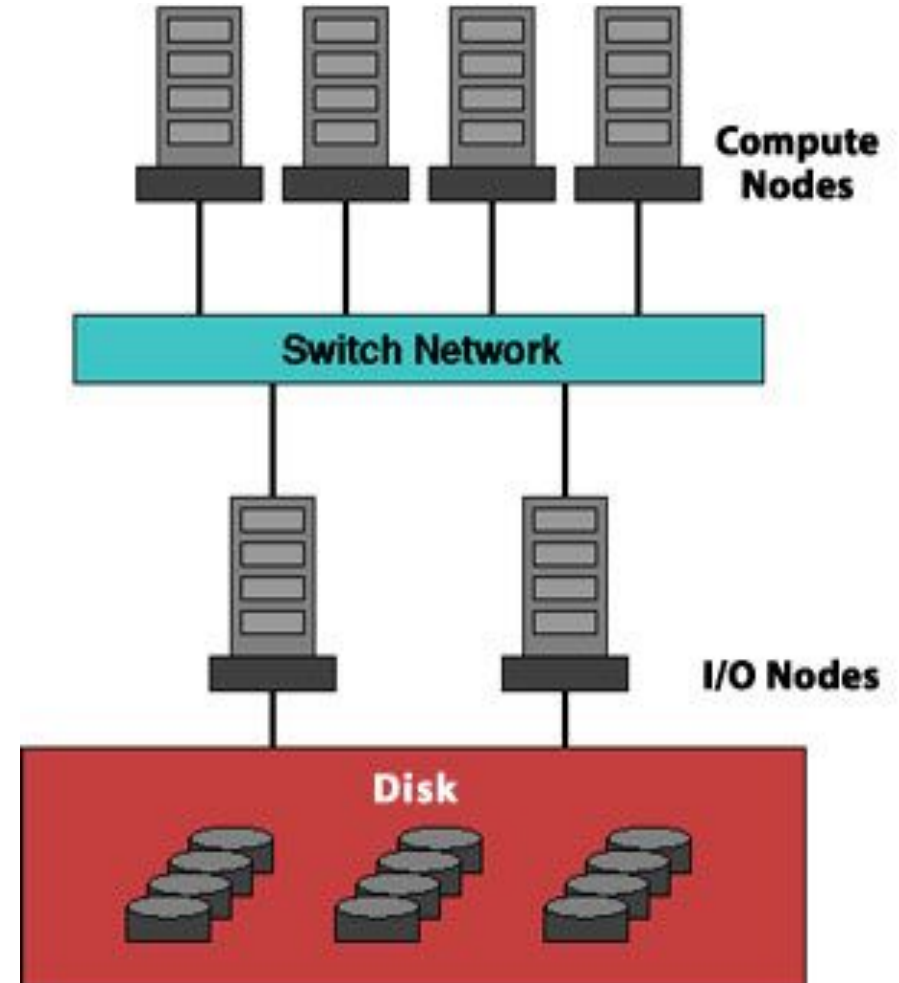
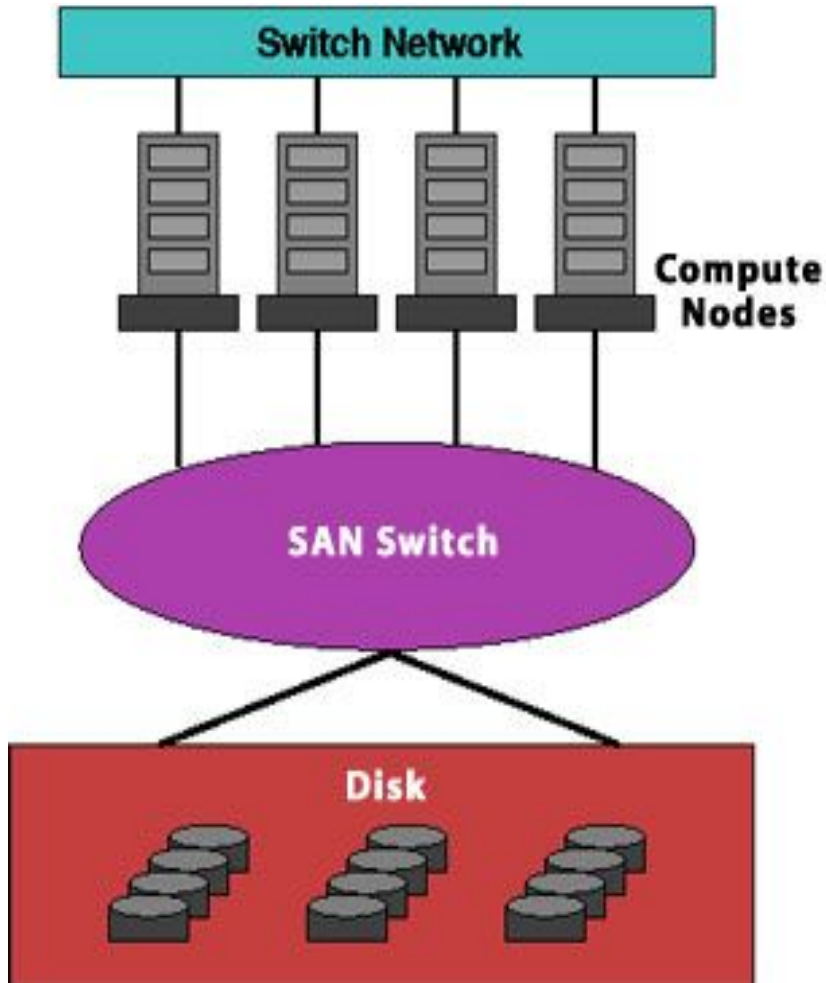
Sistemas de archivos paralelos



- Se requiere un mecanismo de **gestión de metainformación** para encontrar los bloques de los archivos
- Un mismo archivo puede ser accedido simultáneamente desde múltiples nodos de cómputo. Por lo tanto, se requiere un mecanismo de **gestión de bloqueos**
- **Acceso directo a datos** entre los nodos de cómputo y el almacenamiento

Sistemas de archivos paralelos

- Pueden desplegarse utilizando una SAN o nodos de entrada/salida (I/O nodes)



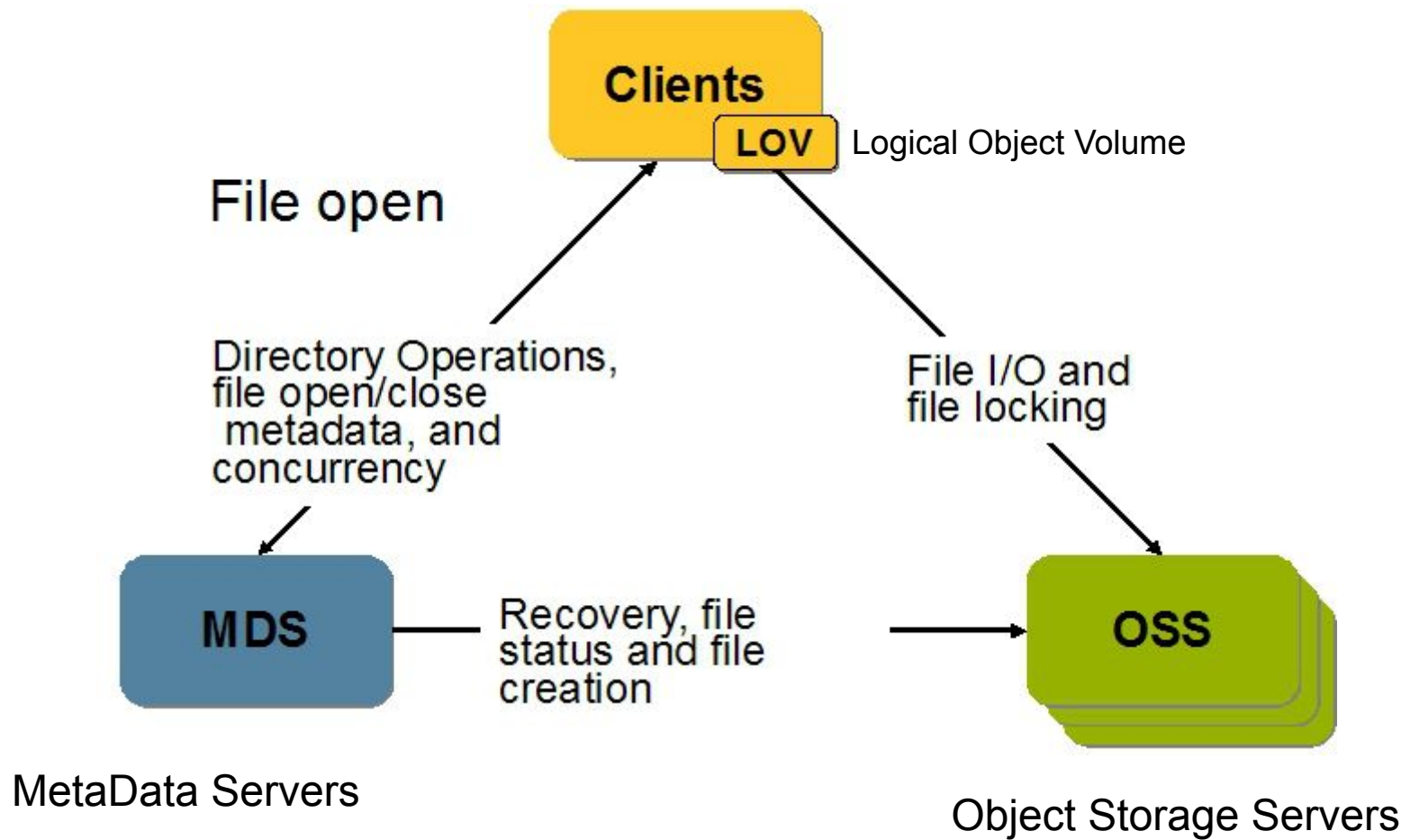
- Son sistemas altamente escalables

Sistemas de archivos paralelos

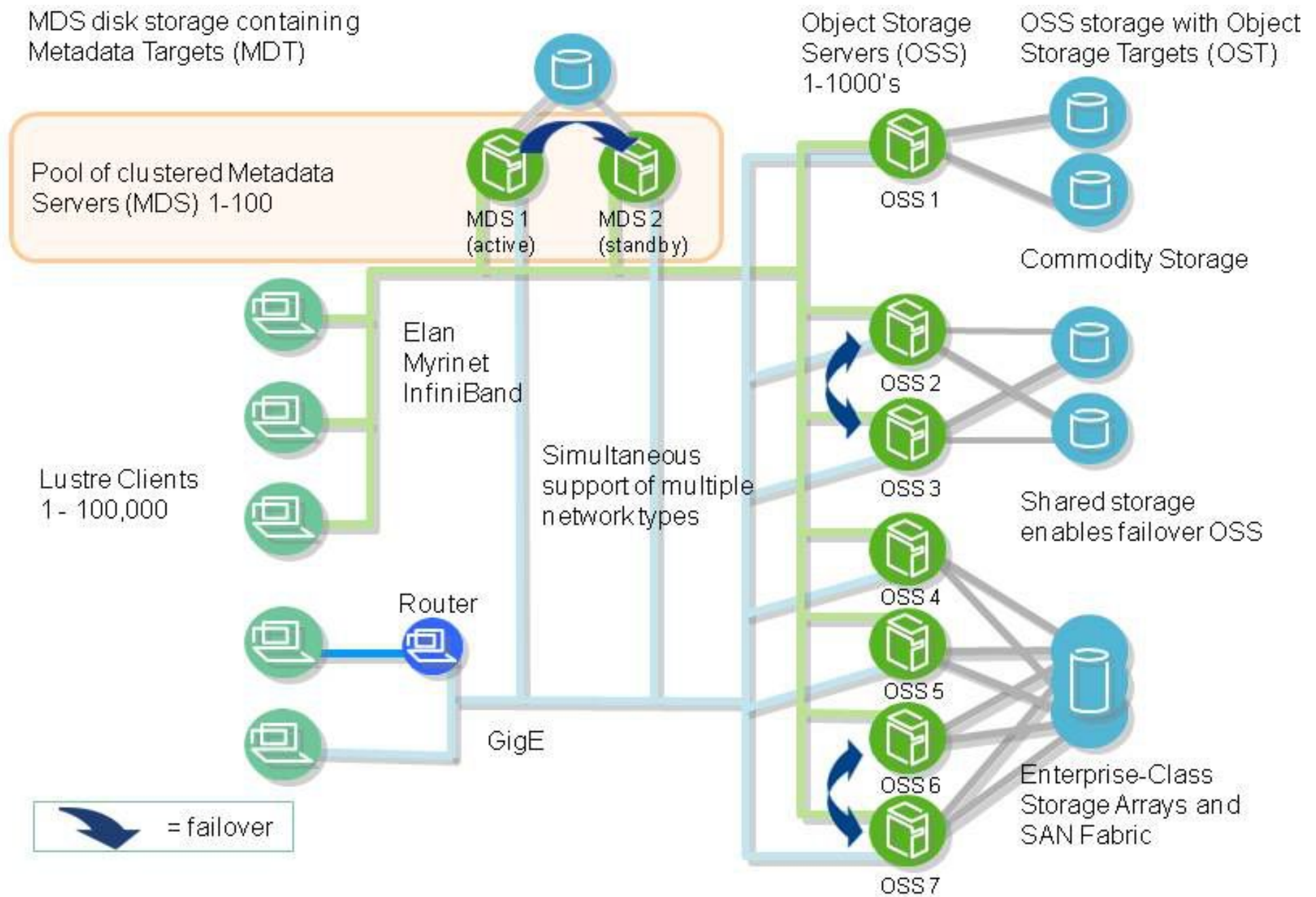
Algunos ejemplos:

- **Lustre**: para Linux, licencia **GPL**. Amplio uso de supercomputadoras del TOP500.
- **PVFS/PVFS2/OrangeFS**: Parallel Virtual File System para Linux (Clemson Univ., Argonne Lab., otros). Licencia **LGPL**.
- **GPFS**: General Parallel File System de IBM.
- **pNFS**: Parallel Network File System es parte del estándar abierto NFS v4.1.
- **Red Hat GFS/GFS2**: Red Hat Global File System. Licencia **GPL**.
- **HP SFS**: HP StorageWorks Scalable File Share. Basado en Lustre.

Lustre



Lustre



Algunas recomendaciones de E/S

- Reducir la E/S tanto como sea posible
- Estudiar el sistema de archivos paralelo específico a utilizar
- Escribir grandes trozos de datos es más eficiente
- Si los archivos son pequeños puede ser mejor no hacer stripping
- Usar la E/S local de los nodos es más eficiente. Por ejemplo: /tmp

Referencias

- Libro de Dongarra
 - 2.2.5 Support for Programming Models
 - 3.1.4 Memory Hierarchy
 - 3.2 Decomposing Programs for Parallelism
 - 3.2.1 Identification of Parallelism
 - 3.2.2 Decomposition Strategy
 - 3.2.3 Programming Models
- Libro de Buyya:
 - 1.5.7 Programming Skeletons
- Libro de Pacheco
 - 1.3 Why we need to write parallel programs
 - 2.3.3 Interconnection networks: Latency and bandwidth
- Libro de Rauber
 - 3.1 Models for Parallel Systems
 - 3.2 Parallelization of Programs
 - 3.3.5 Explicit and Implicit Representation of Parallelism
- Libro de Grama
 - 3.1 Preliminaries
 - 3.3 Characteristics of Tasks and Interactions
 - 3.4 Mapping Techniques for Load Balancing
 - 3.5 Methods for Containing Interaction Overheads
- Documentación específica de productos (NFS, Lustre, etc.)
- Lectura complementaria: Libro de Ian Foster.
 - 2.1 Methodical Design
 - 2.2 Partitioning
 - 2.3 Communication
 - 2.4 Agglomeration
 - 2.5 Mapping